

ELEN90098 REINFORCEMENT LEARNING FOR ENGINEERING
WORKSHOP 1: MULTI-ARMED BANDIT

Department of Electrical and Electronic Engineering
The University of Melbourne

July 7, 2026

Farhad Farokhi (farhad.farokhi@unimelb.edu.au)

Welcome to Workshop 1 for Reinforcement Learning for Engineering!

This workshop introduces you to the multi-arm bandit algorithms that we discussed in the lectures via their implementation in Python. You will investigate the characteristics of these algorithms and evaluate their performance.

Logistics: Be aware that seeking or providing detailed assistance from/to people other than your workshop partner is collusion; see <http://academichonesty.unimelb.edu.au/plagiarism.html> for more information on academic integrity. The workshops are completed in groups of at most 2 students. There will be no additional mark for completing tasks individually.

Submission: Each group must upload two files:

1. A pdf (.pdf) file containing your answers to the questions posed throughout the workshop material. Note that:
 - (a) Only one member of the group needs to upload this file;
 - (b) The naming convention of the file is **Workshop-LastName1-LastName2.pdf** for a group whose members have last names, LastName1 and LastName2.
2. A Python (.py) or Python Notebook (.ipynb) file, containing all the required functions and a main script which calls these functions to generate the required outputs as outlined in the workshop questions. Note that:
 - (a) Only one member of the group needs to upload this file;
 - (b) The naming convention of the file is **Workshop-LastName1-LastName2.py** (or **.ipynb**) for a group whose members have last names, LastName1 and LastName2.

Deadline: 11:59pm on Friday, 4th September.

Notes: You **must** to explain and justify your answers, and to show all the details of your calculations. If you do not provide enough information/detail, you will not receive the allocated marks. **If you do not follow the required naming conventions or you do not upload the needed material, you will receive no mark!** No exception!

Delayed submission: We will not accept any late submission unless the group has asked for an extension prior to the deadline. No exception!

Introduction

Multi-armed bandit is a problem in which a decision maker must iteratively select one of multiple fixed choices (i.e., arms or actions) when the rewards for the choices are not known *a priori* and must be learned with time. The multi-armed bandit problem is a simplified reinforcement learning problem (by removing the time-varying and correlated state) to better understand the impact of exploration versus exploitation (the core issue arising in multi-armed bandit). In this section of the workshop, we will define and launch an experiment using the algorithms presented to you in the lectures for the multi-armed bandit problem. Good luck!

Setting up the arms (initializing the problem)

As the first step, we must setup the arms in the multi-armed bandit experiment. Note that in real-life the arms and their reward structure are setup by nature (e.g., various drugs efficacy) but we are in a laboratory without the luxury of field experiments (they take time, money, and energy in a path peppered by ethical concerns particularly when dealing with human subjects). So we create a fake, or better-said synthetic, environment to investigate various multi-armed bandit algorithms. Starting with synthetic data means we can tweak our codes to make sure there is nothing wrong with our implementation (if you have ever programmed anything, you know the first implementation is often faulty and must be debugged). Try to remember this for a sanity check: Although you yourself is setting up the experimental setup and know the rewards, the algorithms that you implement does not and must not use this knowledge! We must keep faithful to the experiment's premise.

Ok! Let us consider a multi-armed bandit with $k = 10$ arms. We assume that each arm, in our multi-armed bandit, provides a reward that is Gaussian distributed with mean μ_a and variance $\sigma_a = 1.0$ for $a \in \mathcal{A} := \{0, \dots, 9\}$. Therefore, $q_*(a) = \mathbb{E}\{R_t | A_t = a\} = \mu_a$. We assume that $q_*(a) = \mu_a$ are Gaussian distributed with mean 0 and variance 1. Gaussianity is not really important here. We use Gaussian variables as there are libraries in Python for generating Gaussian random variables but you can use anything: uniform, Laplace, Beta, *etc.* The following code snippet shows the way these numbers were generated. Figure 1 illustrates the action values, $q_*(a)$, $a \in \mathcal{A}$. Note that random number generators in computers do not really produce random numbers. They create pseudo¹-random numbers: numbers that look random (they are indistinguishable from random numbers via statistical tests) if you forget the seed that you fed into those functions. Here, I am fixing the seed to be 5 for replicability or reproducibility. I am essentially telling you how I got my exact multi-armed bandit problem. Feel free to change the seed if you like! The distribution of the rewards for each arm are shown in Figure 1 using the so-called violin plot. For any learning method, we can measure its performance and behavior as it improves with experience (or better said data) over 10,000 consecutive time steps. This makes up one run, which is repeated multiple (e.g., 100) times, while fixing the bandit problem, to measure the learning algorithms' average behavior.

¹pseudo means resembling or imitating.

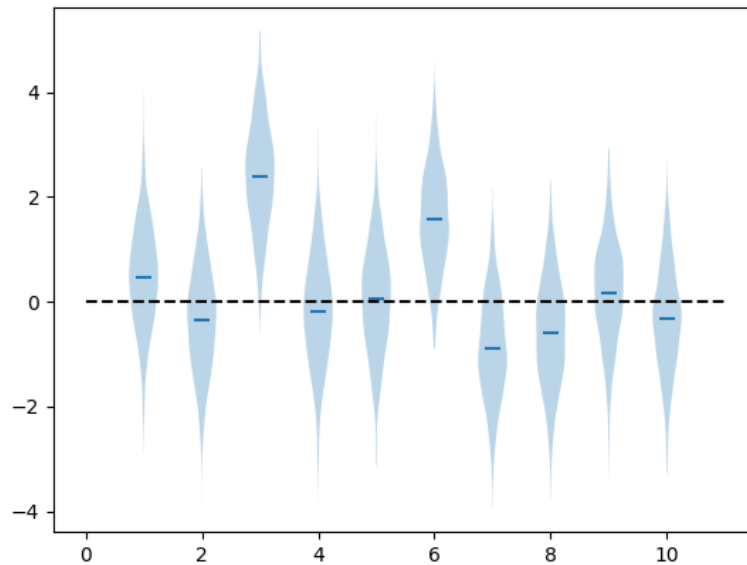


Figure 1: An example of multi-armed bandit with 10 arms. The true value of arm $q_*(a)$ is distributed according to a Gaussian random variable with zero mean and unit variance. The rewards for each arm a is a Gaussian random variable with mean $q_*(a)$ and unit variance, as depicted by violin plots.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 np.random.seed(5)           # fixing randomization seed
5                             # to ensure consistency
6                             # in our experiments
7
8 k=10;                       # number of arms
9
10 q_star=np.random.normal(0,1,k); # average reward in each arm
11 sigma=np.ones(k);           # variance of reward for each arm

```

ϵ -greedy bandits

We start by considering the ϵ -greedy family of multi-armed bandit algorithms. They alternate between playing the greedy optimal arm (exploitation) and randomly playing arms (exploration) to find the best arm. Here, we explore 6 scenarios: 3 scenarios with fixed ϵ and 3 scenarios with time-varying ϵ_t . Based on theoretical analysis, we know fixed ϵ is not good and we must decrease ϵ_t slowly enough to get logarithmic regret. Figure 2 illustrates the expected average reward versus time for all 6 scenarios. It is interesting to note that, while we expect $\epsilon_t \leftarrow \min\{t^{-1/3}(k \log t)^{1/3}, 1\}$ to result in sublinear regret and convergence to the optimal decision as times goes to infinity, its performance in short-time can be rather underwhelming. This points to the asymptotic nature of the regret analysis:

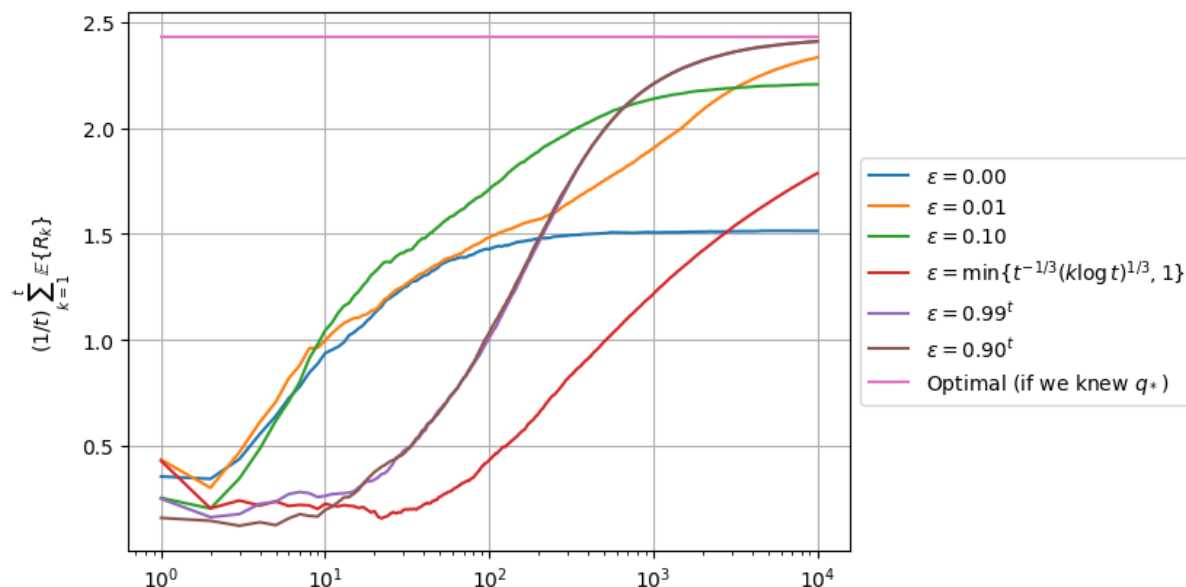


Figure 2: Expected average reward versus time for 6 scenarios: 3 scenarios with fixed ϵ and 3 scenarios with time-varying ϵ_t .

It doesn't tell you much about what the regret is in short time (what does short mean?). In reality, a person or company can go bankrupt or die long before the solutions offered by the multi-armed bandit algorithm converges to the vicinity of the optimal decision. Be careful about asymptotic results!

```

1  # epsilon-greedy bandit
2
3  T=10000;                      # horizon
4  rerun=100;                    # number of reruns
5
6  legend=[];
7  for scenario in range(6):     # we test 6 scenarios:
8                               # 3 with fixed epsilon and
9                               # 3 with time varying epsilon
10
11     R=np.zeros((T,rerun));    # rewards recieved
12     average=np.zeros(T);      # rewards recieved
13
14     match scenario:
15         case 0:                # scenario 0: epsilon fixed to 0
16             epsilon_t=lambda t: 0;
17             legend.append('$\epsilon=0.00$');
18         case 1:                # scenario 1: epsilon fixed to 0.01
19             epsilon_t=lambda t: 0.01;
20             legend.append('$\epsilon=0.01$');
21         case 2:                # scenario 2: epsilon fixed to 0.1
22             epsilon_t=lambda t: 0.1;
23             legend.append('$\epsilon=0.10$');
24         case 3:                # scenario 3: epsilon varying (slides)

```

```

25     epsilon_t=lambda t: min((k*np.log(t+1)/(t+1))**(1/3),1);
26     legend.append('$\epsilon=\min\{(k \log t/t)^{1/3},1\}$');
27 case 4:                                # scenario 4: epsilon slow decrease
28     epsilon_t=lambda t: 0.99**t;
29     legend.append('$\epsilon=0.99t$');
30 case 5:                                # scenario 5: epsilon quick decrease
31     epsilon_t=lambda t: 0.99**t;
32     legend.append('$\epsilon=0.90t$');
33
34 for r in range(0,rerun):
35     Q=np.zeros((k,T+1));               # empirical mean
36     N=np.zeros((k,T+1));               # number of items an arm used
37     A=np.zeros(T,dtype=int);           # arms used
38
39     for t in range(0,T):
40         epsilon=epsilon_t(t);
41
42         if (np.random.uniform()<epsilon):
43             A[t]=int(np.random.randint(0,k));   # exploration
44         else:
45             A[t]=int(np.argmax(Q[:,t]));         #exploitation
46
47         R[t,r]=np.random.normal(q_star[A[t]], sigma[A[t]]);
48
49         for i in range(0,k):               # updating empricial data
50             N[i,t+1]=N[i,t];
51             Q[i,t+1]=Q[i,t];
52             if (A[t]==i):
53                 N[i,t+1]=N[i,t]+1;
54                 Q[i,t+1]=Q[i,t]+(1/N[i,t+1])*(R[t,r]-Q[i,t]);
55
56     for t in range(0,T):
57         average[t]=np.mean(R[0:t,:]);
58
59     plt.semilogx(range(1,T),average[1:T]);
60
61 plt.semilogx([1,T],[np.max(q_star),np.max(q_star)]);
62 legend.append('Optimal (if we knew $q_*$)$');
63
64 plt.grid()
65 plt.legend(legend,loc='center left', bbox_to_anchor=(1, 0.5))
66 plt.ylabel('$\frac{1}{t} \sum_{k=1}^t \mathbb{E}\{R_k\}$')
67
68 plt.show()

```

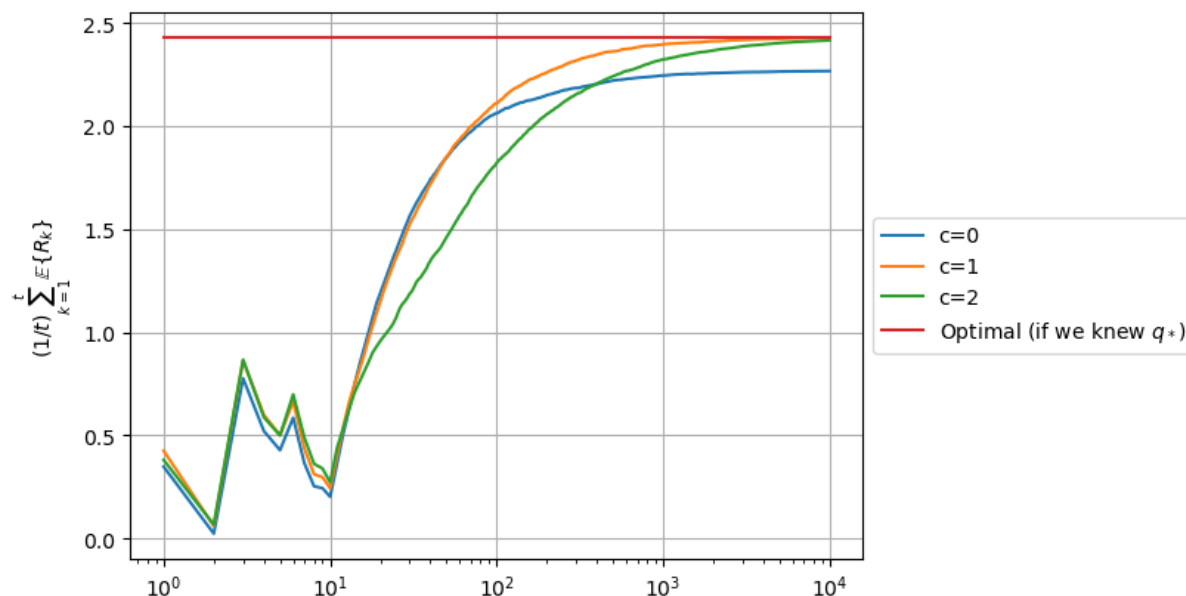


Figure 3: Expected average reward versus time for the Upper-Confidence Bound (UCB) algorithms.

Question

- Q1: Define sublinear regret and explain its significance. [2 marks]
- Q2: Propose a numerical way to examine whether a multi-armed bandit algorithm achieves sublinear regret. [3 marks]
- Q3: Update the provided Python code based on your response to Q2 and explain whether any of the 6 ϵ -greedy scenarios described above and implemented in the provided Python code achieve sublinear regret. [5 marks]

UCB bandits

We now investigate performance of the Upper-Confidence Bound (UCB) algorithms for multi-armed bandit. We particularly explore 3 scenarios: 1 scenario involving greedy decision making with no exploration and 2 scenarios with different balance of exploitation-exploration. Figure 3 illustrates the average expected reward versus time for all scenarios. UCB works perfectly (in two scenarios that involve exploration) and the expected average rewards converges to the optimal solution rapidly.

```

1 # Upper-Confidence Bound (UCB)
2
3 T=10000;                # horizon
4 rerun=100;              # number of reruns
5
6 legend=[];
7 for scenario in range(3):

```

```

8     R=np.zeros((T, rerun));           # rewards recieved
9     average=np.zeros(T);             # rewards recieved
10
11     match scenario:
12         case 0:
13             c=0;
14             legend.append('c=0');
15         case 1:
16             c=1;
17             legend.append('c=1');
18         case 2:
19             c=2;
20             legend.append('c=2');
21
22     for r in range(0, rerun):
23         Q=np.zeros((k, T+1));         # empirical mean
24         N=np.zeros((k, T+1));         # number of items an arm used
25         A=np.zeros(T, dtype=int);     # arms used
26
27         for t in range(0, T):
28             A[t]=int(np.argmax(Q[:,t]+c*np.sqrt(np.log(t+1)/N[:,t])));
29
30             R[t,r]=np.random.normal(q_star[A[t]], sigma[A[t]]);
31
32             for i in range(0, k):
33                 N[i,t+1]=N[i,t];
34                 Q[i,t+1]=Q[i,t];
35                 if (A[t]==i):
36                     N[i,t+1]=N[i,t]+1;
37                     Q[i,t+1]=Q[i,t]+(1/N[i,t+1])*(R[t,r]-Q[i,t]);
38
39         for t in range(0, T):
40             average[t]=np.mean(R[0:t,:]);
41
42     plt.semilogx(range(1, T), average[1:T]);
43
44     plt.semilogx([1, T], [np.max(q_star), np.max(q_star)]);
45     legend.append('Optimal (if we knew $q_*$)');
46
47     plt.grid()
48     plt.legend(legend, loc='center left', bbox_to_anchor=(1, 0.5))
49     plt.ylabel('$\sum_{k=1}^T \mathbb{E}[R_k]$')
50
51     plt.show()

```

Question

Q4: Investigate numerically whether the UCB algorithm learns the true value of each arm over time. [2 marks]

Q5: Numerically compare the regret from the UCB and ϵ -greedy algorithms. [3 marks]

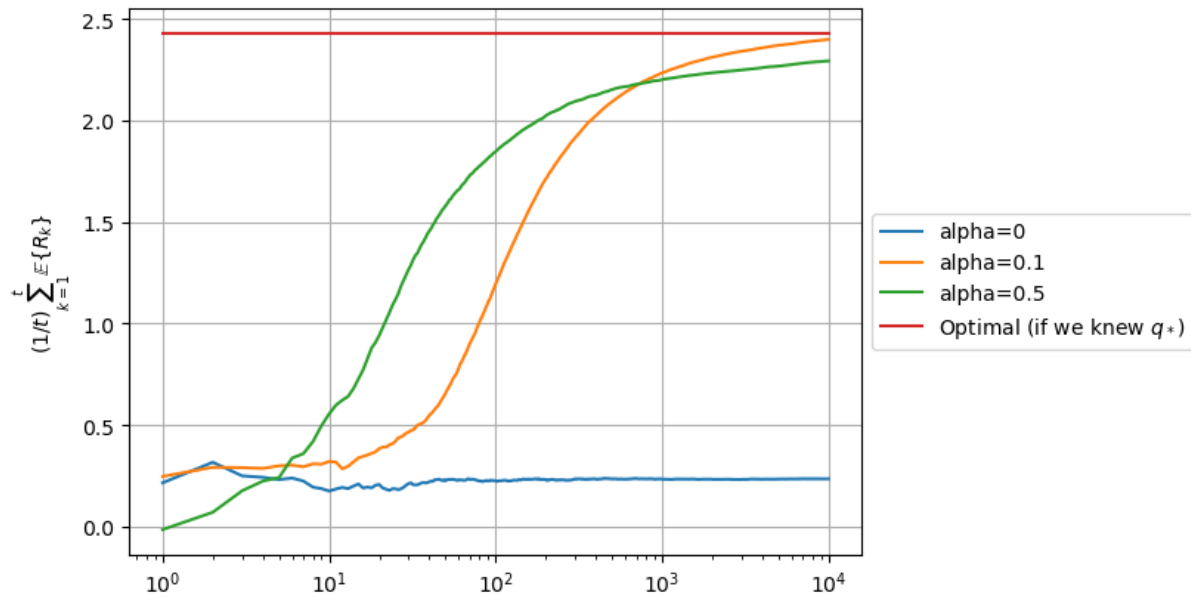


Figure 4: Expected average reward versus time for Upper-Confidence Bound (UCB) algorithms.

Gradient bandits

We now move to implement the gradient bandit algorithm. We explore various scenarios parameterized by the learning rate α in the gradient ascent algorithm. Figure 4 illustrates the expected average reward versus time for all scenarios.

```

1  # Gradient bandit algorithm
2
3  T=10000;                      # horizon
4  rerun=100;                    # number of reruns
5
6  legend=[];
7  for scenario in range(3):
8      R=np.zeros((T,rerun));    # rewards recieved
9      average=np.zeros(T);      # rewards recieved
10
11     match scenario:
12         case 0:                # scenario 0: alpha=0 (no learning)
13             alpha=0;
14             legend.append('alpha=0');
15         case 1:                # scenario 1: alpha=0.1
16             alpha=0.1;
17             legend.append('alpha=0.1');
18         case 2:                # scenario 2: alpha=0.5
19             alpha=0.5;
20             legend.append('alpha=0.5');
21
22     for r in range(0,rerun):
23         H=np.zeros((k,T+1));   # numerical preference

```

```

24     P=np.zeros((k,T+1));           # policy (probability of each action)
25     A=np.zeros(T,dtype=int);       # arms used
26     bR=0;                          # average past rewards (baseline)
27
28     for t in range(0,T):
29         P[:,t]=np.exp(H[:,t])/np.sum(np.exp(H[:,t]));
30
31         A[t]=np.random.choice(range(k), 1, p=P[:,t]);
32
33         R[t,r]=np.random.normal(q_star[A[t]], sigma[A[t]]);
34
35         for i in range(0,k):
36             H[i,t+1]=H[i,t]-alpha*P[i,t]*(R[t,r]-bR);
37             if (A[t]==i):
38                 H[i,t+1]=H[i,t]+alpha*(1-P[i,t])*(R[t,r]-bR);
39
40         bR=bR*t/(t+1)+R[t,r]/(t+1);
41
42     for t in range(0,T):
43         average[t]=np.mean(R[0:t,:]);
44
45     plt.semilogx(range(1,T),average[1:T]);
46
47     plt.semilogx([1,T],[np.max(q_star),np.max(q_star)]);
48     legend.append('Optimal (if we knew $q_*$)');
49
50     plt.grid()
51     plt.legend(legend,loc='center left', bbox_to_anchor=(1, 0.5))
52     plt.ylabel('$\frac{1}{t} \sum_{k=1}^t \mathbb{E}\{R_k\}$')
53
54     plt.show()

```

Question

Q6: Numerically compare the regret from the gradient bandit and UCB algorithms. [3 marks]

Q7: Does the gradient bandit algorithm converge for all choice of $\alpha > 0$? [2 marks]